

Using Fourier Transforms to Classify Music Genres

Paulina Huang
Math 104, Winter 2025
Stanford University
paulina5@stanford.edu

I. INTRODUCTION

How can music streaming platforms (e.g., Spotify, Apple Music, etc.) populate your listening queues with similar songs, or curate playlists based on the different genres of music you listen to? While many algorithms are at play to keep you listening as long as possible, this project will focus on an algorithm that can classify the genres of music.wav files. Indeed, it has been confirmed that applications like Spotify use extremely sophisticated machine-learning algorithms that can organize the roughly 100,000 new tracks uploaded per day[1, 2].

This project investigates the practical application of Fourier Transformations and machine learning to automate genre classification. Using the Short-Time Fourier Transform (STFT) to generate spectrograms, we decompose audio signals into their time-frequency components, capturing distinctive patterns across genres such as blues, rock, and classical. These spectrograms, combined with derived features like Mel-Frequency Cepstral Coefficients (MFCCs) and spectral centroids, serve as inputs to a supervised learning model designed to map acoustic signatures to genre labels. While simplified compared to

industrial-scale systems, this approach mirrors the core principles behind platforms like Spotify.

II. DATA SET AND SAMPLING

The GTZAN dataset was sourced from Kaggle and originally curated by George Tzanetakis in 2001. It consists of 1,000 audio tracks, evenly distributed across 10 genres: blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae, and rock. Each genre contains 30-second .wav files sampled at 22,050 Hz and stored in mono format, ensuring high-fidelity analysis of time-frequency patterns. The dataset is organized into genre-specific subdirectories (e.g., blues/, rock/), simplifying systematic access for feature extraction[3].

Despite its widespread use, it is important to note that GTZAN has limitations: its age restricts the representation of modern subgenres, some files are corrupted or mislabeled, and its reliance on commercial recordings introduces a bias toward Western musical traditions.

III. METHODOLOGY

A. Audio Preprocessing with Librosa and Short-Time Fourier Transform

To re-standardize inputs (to account for corrupted or mislabeled files), I used Python’s Librosa library, which provides functions to quickly extract key audio features and metrics from audio files, to standardize the .wav files in the GTZAN data set (22.05 kHz sample rate, mono channel).

Then, I generated a spectrogram for each .wav file in the data set using the Short-Time Fourier Transform (STFT). A spectrogram is a “visual way of representing the signal strength, or “loudness”, of a signal over time at various frequencies present in a particular waveform.”

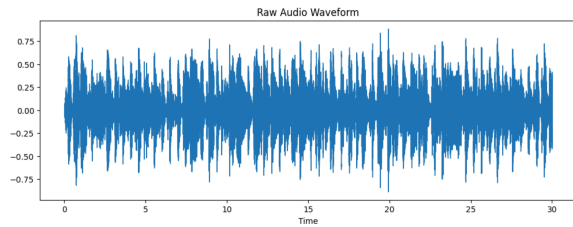


Fig. 1: Visual representation of the raw audio waveform of blues.00000.wav

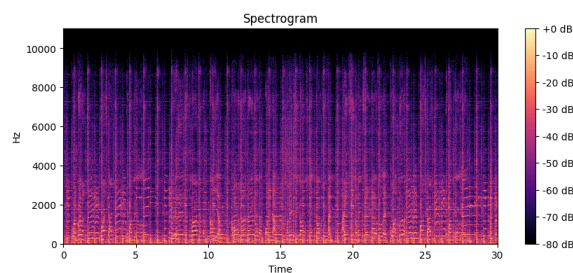


Fig. 2: Spectrogram generated from the raw blues.00000.wav file

In particular, the STFT is able to extract this data from the .wav file by transforming the given time-domain signals to a frequency domain. This is achieved by dividing the audio file into

smaller segments and then applying the Discrete Fourier Transform (DFT) to each, hence, “Short-Time.” This segmentation is crucial, especially when working with longer audio samples, as the division allows us to represent the frequency content over time versus one static moment.

The Fourier Transform itself is derived from the Fourier Basis

$$\left\{ \frac{1}{\sqrt{2}}, \quad \cos\left(\frac{2\pi kx}{L}\right), \quad \sin\left(\frac{2\pi kx}{L}\right) \right\}$$

a basis for any continuous function $f(x)$ on $[0, L]$, such that $f(x)$ can be expressed as a Fourier series using an infinite sum of sinusoids:

$$[f(x) = \sum_{k=-\infty}^{\infty} c_k e^{i2\pi kx/L}]$$

where k is the frequency index, $e^{i2\pi kx/L} = \cos(2\pi kx / L) + i\sin(2\pi kx / L)$ are the basis functions. In short, Fourier analysis is essentially a change of basis where we express a function in terms of complex exponentials. The Fourier Transform generalizes the Fourier series to non-periodic functions, allowing us to decompose any function $f(x)$ into its frequency components:

$$[F(\xi) = \int_{-\infty}^{\infty} f(x) e^{-i2\pi \xi x} dx]$$

where $F(\xi)$ gives the amplitude of the frequency component ξ . Because this transformation is a linear operator, the

Fourier Transform can be seen as an infinite-dimensional version of a matrix multiplication. In reality, most of the signals that we want to process are discrete, not continuous, so we use DFT which is computed using a finite summation as opposed to an infinite one. As discussed previously, implementing DFT in segments results in the STFT matrix, where the columns of the resulting matrix represent the sound's frequency, the rows represent the time, and the magnitude of a single element provides an "estimate of the energy in a certain frequency band at a certain time position in the input signal." The spectrograms generated from STFT not only provide information about the audio's frequency at given time intervals but are also fundamental to determining other key audio features used in genre classification[4].

B. Building the Feature Vector

Computing an accurate feature vector for each audio sample required me to extract a few other key features, such as the Mel-Frequency Cepstral Coefficients (MFCC), Spectral Centroid, and Zero Crossing Rate, to capture more distinctions that the spectrogram may have missed.

The MFCCs capture the timbral/textural characteristics (the distinct "aesthetic" qualities of a sound that allow you to differentiate, for example, different instruments) of an audio sample[5]. The coefficients can be calculated by applying the Mel Filterbank matrix, a set of triangular filters, to the power spectrum to simulate human hearing. The power

spectrum refers to the magnitude squared of the previously obtained STFT matrix. Finally, the coefficients can be extracted by applying the Discrete Cosine Transform (DCT) which de-correlates the filterbank energies (similar to PCA). The number of MFCCs extracted can determine the timbre/texture of the audio[6].

The Spectral Centroid measures the "center of mass" of the frequency spectrum and can be thought of as the "brightness" of the sound. The spectral centroid is computed as a weighted average of frequencies, where the weights are the magnitudes of the frequency bins:

$$\text{Spectral Centroid} = \frac{\sum_k f_k \cdot |X_k|}{\sum_k |X_k|}$$

Here, f_k is the frequency vector, and $|X_k|$ is the magnitude spectrum vector, also obtained through the Fourier transform[7].

Finally, the Zero Crossing Rate measures the "noisiness" of an audio signal by calculating how often the audio signal crosses zero. While Python packages like Librosa and NumPy calculate the rate with vectorized operations, the Zero Crossing Rate essentially counts the total number of the wave's sign changes[8].

These characteristics (along with several others, discussed later) made up the 1D array, or feature vector, assigned to each audio file. In total this produced a 1,055-dimensional vector, of which 1,025 dimensions came from the time-averaged spectrogram alone. The engineered

features discussed above therefore accounted for under 3% of the input, a design choice I return to in Section IV.

C. Training a Classifier

After initially trying to play around with a RandomForest classifier, I found that the model could not achieve an accuracy above 56%.

Thus, I ended up going with XGBoost, which stands for Extreme Gradient Boosting, to build my classifier. Put simply, XGBoost evaluates a tree of if-then-else true/false “feature questions” and then classifies the audio samples based on what it perceives to be the minimum number of questions asked to make an accurate prediction.

IV. RESULTS AND DISCUSSION

I was able to increase the initial 56% tested accuracy from the RandomForest classifier to 64% with the switch to XGBoost. This is likely because gradient boosting corrects its own errors iteratively, fitting each new tree to the residuals of the ensemble so far, which suits a feature space where no single feature is strongly discriminative on its own.

In my attempt to further increase accuracy, I also applied the Synthetic Minority Oversampling Technique (SMOTE) [9]. In retrospect, this was unnecessary: GTZAN is balanced by construction at 100 tracks per genre, and only one corrupt file was skipped, so the resampling had almost nothing to correct. I also added four further features not discussed above — spectral bandwidth,

spectral rolloff, chroma, and tempo — and used GridSearchCV to tune the model's hyperparameters (tree count, depth, learning rate, and subsampling ratio) over 5-fold cross-validation. Finally, I reduced the dimensionality of my data using PCA, retaining 95% of the variance, which brought total processing time from over 10 minutes down to 6.

It is also worth noting that 64% sits below the 70–80% typically reported on GTZAN using comparable feature-based methods. The most likely explanation is the imbalance in the feature vector described in Section III.B: with 1,025 of 1,055 dimensions coming from the time-averaged spectrogram, the features I selected specifically to discriminate between genres (the MFCCs, spectral centroid, and zero crossing rate) were diluted to under 3% of the model's input. Dropping the averaged spectrogram and retaining only the engineered features would be the first experiment I would run given more time.

After fully executing the project, doing some more research, and discussing the project with my brother (a CS and Applied Math major at Johns Hopkins), it sounds like a Support Vector Machine (SVM) and/or Neural Network would have been more ideal for the type of data I was working with. Implementing these changes could be a future project (with a heightened understanding of machine learning).

V. EXTENSIONS

These types of audio-classifying algorithms don't have to be applied only to

music genres. The classification of musical instruments and timbral/textural characteristics was discussed along with MFCCs. Looking back, solely focusing on MFCCs might have been more manageable given my skill set.

Outside of music applications, this method can also be applied to distinguishing language samples from one another or identifying speech tones. With enough samples and parameters (and better machine learning), this type of algorithm can probably differentiate almost any “bucket” of audio into distinct categories.

VI. FUTURE RESEARCH

Returning to music streaming apps, I also think that it would be a cool project idea to learn how prediction matrices work on platforms like Spotify. In particular, when the app prompts new songs in your queue, how the Spotify DJ works, or how personalized playlists (like Daylist and Discover Weekly) are curated.

VII. REFERENCES

- [1] Stackpole, Beth. 2022. “How Wayfair and Spotify use machine learning to engage customers.” MIT Management: Sloane School.
<https://mitsloan.mit.edu/ideas-made-to-matter/how-wayfair-and-spotify-use-machine-learning-to-engage-customers>.
- [2] Peoples, Glenn. 2023. “The Ledger: Are There Really 100,000 New Songs Uploaded a Day? Maybe More.” Billboard Pro.
<https://www.billboard.com/pro/how-much-music-added-spotify-streaming-services-daily/>.
- [3] “GTZAN Dataset - Music Genre Classification.” 2020. Kaggle.
<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>.
- [4] “How do I apply a binary mask and STFT to produce an audio file?” 2018. Stack Exchange.
<https://dsp.stackexchange.com/questions/50964/how-do-i-apply-a-binary-mask-and-stft-to-produce-an-audio-file>.
- [5] Bond, Eddie. 2022. “Timbre, texture, and tone: three concepts of sound.” Skoove.
<https://www.skoove.com/blog/timbre-texture-and-tone-in-music/>.
- [6] “MFCC Technique for Speech Recognition.” 2023. Analytics Vidhya.
<https://www.analyticsvidhya.com/blog/2021/06/mfcc-technique-for-speech-recognition/>.
- [7] Kulkarni, Nilesh, and Vinayak Bairagi. 2018. “Use of Complexity Features for Diagnosis of Alzheimer's Disease.” In EEG-Based Diagnosis of Alzheimer Disease: A Review and Novel Approaches for Feature Extraction and Classification Techniques, 47-59. N.p.: Elsevier. Science.
<https://www.sciencedirect.com/science/article/pii/B9780128153925000046>.
- [8] “Pre-processing and feature extraction.” 2021. In Biosignal Processing and Classification Using Computational Learning and Intelligence: Principles, Algorithms, and Applications, edited by Alejandro A. Torres-García, Carlos A. Reyes Garcia, Luis Villasenor-Pineda, and Omar Mendoza-Montoya, 59-91. N.p.: Elsevier Science.
<https://www.sciencedirect.com/science/article/pii/B9780128201251000142>.

[9] Brownlee, Jason. 2021. "SMOTE for Imbalanced Classification with Python." Machine Learning Mastery.
<https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/>.